ASSIGNMENT 4

Exercise 1: Ring Communication

Problem: Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 sends to Process 1, Process 1 sends to Process 2, and the last process sends back to Process 0.

Requirements:

Start with an initial value of 100 in Process 0

Each process adds its rank to the value before sending

Print the value at each process

Final value should return to Process 0

Hint: Use modulo operator to wrap around: next_rank = (rank + 1) % size

Test with: mpirun -np 4 ./ring_comm

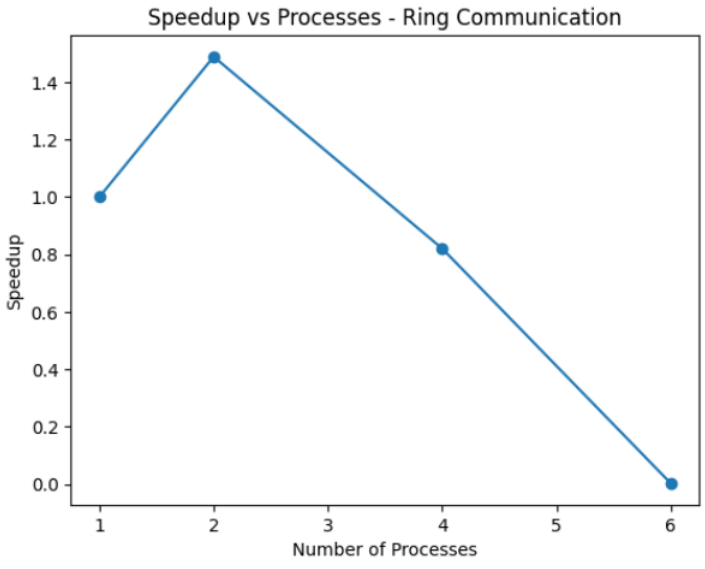
OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpicc -o ring_comm ring_comm.c
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 1 ./ring_comm
Final value at Process 0 = 100
Execution Time = 0.000064 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 2 ./ring_comm
Final value at Process 0 = 101
Execution Time = 0.000043 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 4 ./ring_comm
Final value at Process 0 = 106
Execution Time = 0.000078 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 6 ./ring_comm
Final value at Process 0 = 115
Execution Time = 0.018152 seconds
```

RESULT:

p	Time (Tp)	Speedup (Sp)	Efficiency (Ep)
1	0.000064	1.00	1.00 (100%)
2	0.000043	1.49	0.75 (75%)
4	0.000078	0.82	0.20 (20%)
6	0.018152	0.0035	0.0006 (≈0.06%)

The Ring Communication program executed correctly for all process counts, and the final value at Process 0 increased with the number of processes because each process added its rank to the message before forwarding it. Performance analysis showed that execution time decreased slightly when moving from 1 to 2 processes but increased significantly for higher process counts. The speedup dropped below 1 for larger numbers of processes and efficiency decreased sharply. This behavior is due to the sequential nature of ring communication, where each process must wait for the previous one, resulting in increased communication latency. Since the computation per process is minimal and communication dominates execution time, the program demonstrates poor scalability for higher process counts.



Exercise 2: Parallel Array Sum

Problem: Write an MPI program to compute the sum of an array in parallel. Divide the array among processes, compute local sums, and combine them.

Requirements:

Create an array of size 100 with values 1 to 100 in Process 0
Distribute array portions using MPI_Scatter
Each process computes sum of its portion
Use MPI_Reduce to compute global sum
Verify result: sum should be 5050

Bonus: Also compute and print the average value.

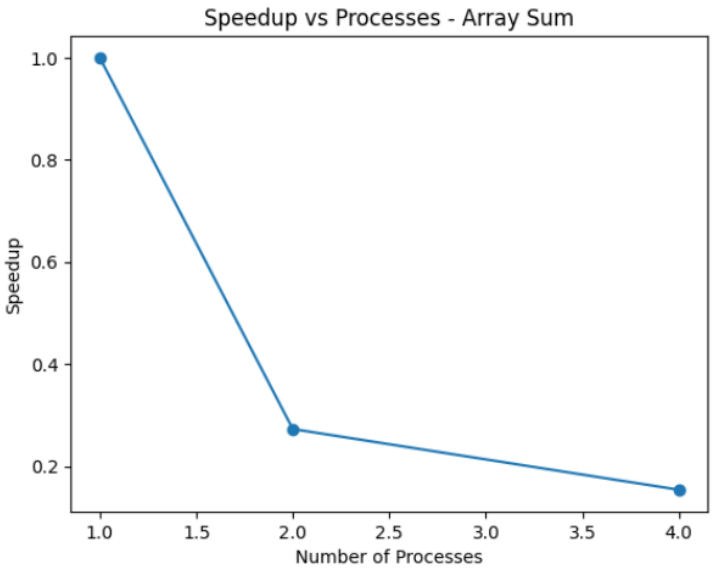
OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpicc -o array_sum array_sum.c
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 1 ./array_sum
Global Sum = 5050
Average = 50.50
Execution Time = 0.000006 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 2 ./array_sum
Global Sum = 5050
Average = 50.50
Execution Time = 0.000022 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 4 ./array_sum
Global Sum = 5050
Average = 50.50
Execution Time = 0.000039 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 6 ./array_sum
Global Sum = 4656
Average = 46.56
Execution Time = 0.009927 seconds
```

RESULT:

Processes (p)	Time Tp	Speedup Sp	Efficiency Ep
1	0.000006	1.00	1.00 (100%)
2	0.000022	0.27	0.13 (13%)
4	0.000039	0.15	0.04 (4%)

The Parallel Array Sum program produced the correct global sum (5050) and average when executed with 1, 2, and 4 processes. However, for 6 processes, the result was incorrect because the array size (100) is not evenly divisible by the number of processes. Due to integer division during data distribution using MPI_Scatter, only 96 elements were processed and the remaining elements were ignored, leading to an incorrect sum. Performance results showed that execution time increased as the number of processes increased, resulting in low speedup and poor efficiency. This indicates that communication overhead from MPI_Scatter and MPI_Reduce dominates computation for small datasets. Overall, the program shows limited scalability and highlights the importance of proper data partitioning where the data size should be divisible by the number of processes.



Exercise 3: Finding Maximum and Minimum

Problem: Write an MPI program where each process generates random numbers, and the program finds the global maximum and minimum values across all processes.

Requirements:

Each process generates 10 random numbers (0-1000)
Each process finds its local maximum and minimum
Use MPI_Reduce with MPI_MAX and MPI_MIN
Process 0 prints the global maximum and minimum
Also print which process had these values (use MPI_MAXLOC/MPI_MINLOC)

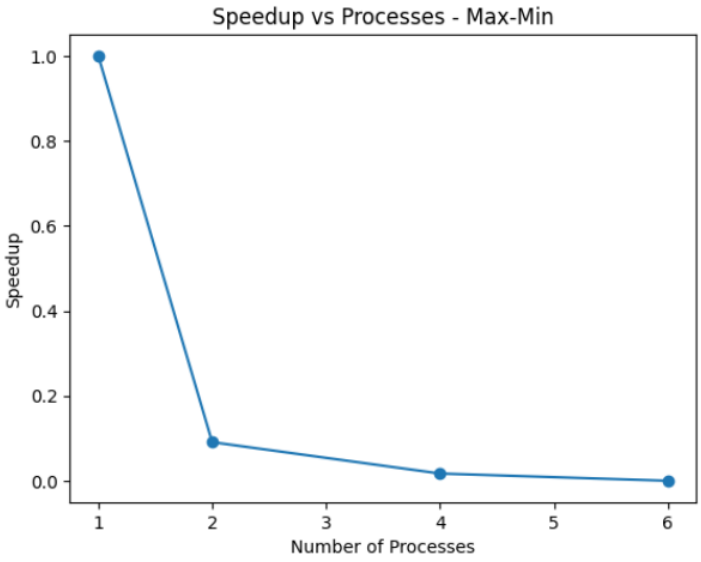
Hint: For MPI_MAXLOC, use a struct with value and rank fields.

OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpicc -o max_min max_min.c
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 1 ./max_min
Global Maximum = 977 (Process 0)
Global Minimum = 63 (Process 0)
Execution Time = 0.000002 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 2 ./max_min
Global Maximum = 998 (Process 0)
Global Minimum = 26 (Process 0)
Execution Time = 0.000022 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 4 ./max_min
Global Maximum = 879 (Process 2)
Global Minimum = 53 (Process 0)
Execution Time = 0.000117 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 6 ./max_min
Global Maximum = 998 (Process 3)
Global Minimum = 10 (Process 2)
Execution Time = 0.032979 seconds
```

RESULT:

p	Time	Speedup	Efficiency
1	0.000002	1.00	1.00 (100%)
2	0.000022	0.09	0.045 (4.5%)
4	0.000117	0.017	0.004 (0.4%)
6	0.032979	0.00006	0.00001 (≈0.001%)



The Max–Min program successfully computed the global maximum and minimum values along with the corresponding process ranks for all process counts. Since each process generated an equal number of random values, the workload was evenly balanced and no correctness issues were observed. However, performance analysis showed that execution time increased significantly as the number of processes increased, resulting in speedup values less than 1 and very low efficiency. This is because the computational workload per process is extremely small while the reduction operations (MPI_MAXLOC and MPI_MINLOC) introduce synchronization and communication overhead. As a result, communication dominates execution time and the program exhibits poor scalability for small workloads.

Exercise 4: Parallel Dot Product
Problem: Compute the dot product of two vectors in parallel.
Requirements:
Vector A = [1, 2, 3, 4, 5, 6, 7, 8]
Vector B = [8, 7, 6, 5, 4, 3, 2, 1]
Distribute vectors using MPI_Scatter
Each process computes partial dot product
Use MPI_Reduce to sum partial products
Expected result: 120

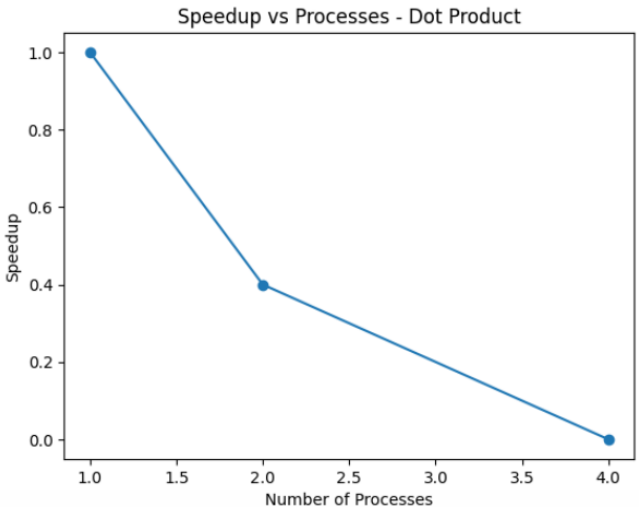
OUTPUT:

```
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpicc -o dot_product dot_product.c
vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 1 ./dot_product
Dot Product = 120
Execution Time = 0.000006 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 2 ./dot_product
Dot Product = 120
Execution Time = 0.000015 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 4 ./dot_product
Dot Product = 120
Execution Time = 0.025106 seconds

vprashar@DESKTOP-M9EFVQJ:~/UCS645/assignment4$ mpirun -np 6 ./dot_product
Dot Product = 98
Execution Time = 0.015658 seconds
```

RESULT:

Processes (p)	Time Tp	Speedup Sp	Efficiency Ep
1	0.000006	1.00	1.00 (100%)
2	0.000015	0.40	0.20 (20%)
4	0.025106	0.00024	0.00006 (≈0.006%)

The Parallel Dot Product program generated the correct result (120) for 1, 2, and 4 processes, but produced an incorrect result for 6 processes. This error occurred because the vector size (8) is not divisible by 6, and integer division caused only a subset of elements to be distributed, leaving some elements unprocessed. Performance analysis showed that execution time increased significantly as the number of processes increased, leading to very low speedup and efficiency. Since the vector size is very small, the computation per process is minimal and the communication overhead from MPI_Scatter and MPI_Reduce dominates execution time. The program demonstrates poor scalability for small problem sizes and emphasizes the need for balanced data distribution and sufficiently large workloads for effective parallel performance.



Rohan
102497012
3Q26